Historias de Usuario:

- Nosotros decidimos implementar 3 de las 8 historias de usuario del banco, las cuales fueron: SOC-01 , ADM-01 y ADM-03. Estas las escogimos dado que en nuestro código original del Proyecto 1 , las bases de estas ya estaban casi que totalmente hechas.

ID	Rol	Historia	Elegida	Criterios de Aceptación
ADM-01	Administrador	Trasladar un producto entre las tiendas del club	X	3
ADM-02	Administrador	Revisar solicitudes de cambio de turno		2
ADM-03	Administrador	Aprobar o rechazar sugerencias de nuevos artículos	X	3
ENT-01	Entrenador	Definir la alineación para un partido oficial		3
ENT-02	Entrenador	Verificar el mínimo de jugador habilitados		3
JUG-01	Jugador	Reservar una cancha o mesa de deporte individual		3
SOC-01	Socio	Reservar una instalación para practica individual	X	3
SOC-02	Socio	Inscribirse en equipo de conjunto		2

Trazabilidad:

- Cada criterio de aceptación de las historias de usuario elegidas tiene su propia prueba. Todas las pruebas pasan

HU	Criterios de Aceptación	Qué se verifica	Clase y Método	Prueba
SOC-01	**CA-1**	Hay cupo, se registra la reserva	Club.reservar -- DisciplinaIndividual.registrarReserva	ReservaTest.reservaCuandoHayCupos
SOC-01	**CA-2**	Sin instalaciones libres, se rechaza la reserva	DisciplinaIndividual.registrarReserva	ReservaTest.rechazarCuandoNoHayCupo
SOC-01	**CA-3**	Pádel con 2 jugadores, se rechaza	Constructor de Reserva	ReservaTest.rechazarPadelConDosJugadores
ADM-01	**CA-1**	El producto pasa de la tienda origen a la de destino	Club.trasladarProducto	TrasladoTest.testTrasladoExitoso
ADM-01	**CA-2**	No está en origen, no se traslada	Club.trasladarProducto	TrasladoTest.testFallaProductoNoExisteEnOrigen
ADM-01	**CA-3**	Ya está en el destino, no se traslada	Club.trasladarProducto	TrasladoTest.testFallaProductoYaExisteEnDestino
ADM-03	**CA-1**	Snack con alérgenos, se aprueba y entra al catálogo	Club.aprobarSugerencia -- SugerenciaProducto.crearProducto	SugerenciaTest.apruebaSnackConAlergenosYCreaProducto
ADM-03	**CA-2**	Snack sin alérgenos declarados, no se aprueba	Club.aprobarSugerencia	SugerenciaTest.noApruebaSnackSinAlergenos

ADM-03	**CA-3**	Solicitud rechazada, el empleado la ve y no está en el catálogo	Club.rechazarSugerencia -- Club.sugerenciasDe	SugerenciaTest.sugerenciaRechazadaVisibleParaEmpleado
T-SOC-01	**CA-1**	Socio estándar se inscribe y baja el cupo estándar	Club.inscribirTorneo	TorneoTest.testInscripcionSocioEstandar
T-SOC-01	**CA-2**	Socio con buen ranking ocupa cupo preferencial	Club.inscribirTorneo	TorneoTest.testInscripcionSocioPreferencial
T-SOC-04	**CA-2**	Con bono, el total baja según el porcentaje y el bono queda usado	Club.comprar -- Venta.aplicarBono	BonoTest.comprarConBonoAplicadoDescuento
T-SOC-04	**CA-2**	Sin usar el bono, se paga el precio completo y el bono se conserva	Club.comprar	BonoTest.compraSinBonoNoLoUsa
T-SOC-04	**CA-3**	Un bono usado no se puede volver a usar	Club.comprar -- BonoDeDescuento.marcarUsado	BonoTest.bonoSoloSeUsaUnaVez
T-SOC-04	**CA-4**	No se acumula el bono con puntos	Venta.aplicarBono -- Venta.redimirPuntos	BonoTest.bonoNoAcumulableConPuntos
T-SOC-04	**Unitaria**	El porcentaje de bono debe ser válido	Constructor de BonoDeDescuento	BonoTest.rechazaPorcentajeInvalido

T-EMP-01	**CA-1**	**Empleado fuera de turno se inscribe gratis y sin derecho a premio**	**Club.inscribirEmpleadoEnTorneo -- Torneo.inscribirEmpleado**	**PersistenciaTest.empleadosLoginTurnosYRechazos**
T-EMP-01	**CA-2**	**Empleado con turno superpuesto no puede inscribirse**	**Empleado.coincideConTurno -- Club.inscribirEmpleadoEnTorneo**	**PersistenciaTest.empleadosLoginTurnosYRechazos**
T-EMP-01	**CA-3**	**Empleado ya inscrito no puede inscribirse de nuevo**	**Torneo.inscribirEmpleado -- Club.inscribirEmpleadoEnTorneo**	**PersistenciaTest.empleadosLoginTurnosYRechazos**
T-EMP-01	**CA-4**	**La inscripción conserva la condición gratuita y sin premio**	**InscripcionTorneoEmpleado -- PersistenciaClub.guardar/cargar**	**PersistenciaTest.empleadosLoginTurnosYRechazos**

Historias de Usuario: Torneos del Administrador

Las historias de usuario del administrador describen las funcionalidades necesarias para gestionar el ciclo de vida completo de un torneo dentro del club, desde su creación hasta la finalización y entrega de premios.

HU-T-ADM-01: Crear un nuevo torneo individual
Como administrador, quiero crear un torneo individual con sus reglas, cupos y horarios, para ofrecer nuevas opciones competitivas o recreativas a los socios y empleados del club.

Criterios de aceptación:

- CA-1: Dado un torneo que aún no existe en el sistema, cuando el administrador ingresa un ID único y los datos correspondientes, entonces el sistema crea el torneo en estado ABIERTAS y lo agrega a la lista del club.
- CA-2: Dado un intento de creación de torneo, cuando el administrador ingresa un ID que ya pertenece a otro torneo, entonces el sistema rechaza la operación e informa que el ID ya existe.
- CA-3: Dado un torneo que se está creando como RECREATIVO, cuando el administrador lo registra, entonces el sistema le asigna una tarifa de 0.0 automáticamente, ignorando cualquier valor ingresado.

Entradas: ID único del torneo, nombre, tipo (Recreativo/Competitivo), modalidad (Individual), cupo máximo, tarifa, fecha, hora de inicio y hora de fin.
Salidas: Mensaje de confirmación de creación exitosa o mensaje de rechazo con el motivo del fallo.

Cumplimiento de INVEST: La historia es Independiente ya que crear el torneo no depende de que existan inscritos. Es Negociable porque los datos que se solicitan en la consola se pueden adaptar. Aporta Valor al permitir la gestión operativa del club. Es Estimable y Pequeña ya que se limita a la inicialización de la entidad, y es Comprobable mediante pruebas de validación de ID y asignación de tarifas.

Clases y métodos relacionados: La operación se ejecuta en ConsolaAdministrador, que solicita los datos por consola y crea la instancia con new Torneo(...). La programación se gestiona con Torneo.programar() y finalmente se añade al sistema con Club.agregarTorneo().

HU-T-ADM-02: Iniciar un torneo
Como administrador, quiero cambiar el estado de un torneo para iniciar sus encuentros, asegurando que ya no se reciban más inscripciones y los participantes puedan comenzar a jugar.

Criterios de aceptación:

- CA-1: Dado un torneo en estado ABIERTAS, cuando el administrador decide iniciarlo, entonces el sistema cambia el estado del torneo a EN_CURSO y cierra las inscripciones.
- CA-2: Dado un torneo que ya se encuentra en estado EN_CURSO o FINALIZADO, cuando el administrador intenta iniciarlo de nuevo, entonces el sistema rechaza la acción por estado inválido.

Entradas: Identificador del torneo.
Salidas: Confirmación de que el torneo fue iniciado o mensaje de error por estado inválido.

Cumplimiento de INVEST: Es Independiente de los partidos que se jueguen, aporta Valor bloqueando nuevas inscripciones y es Comprobable revisando que el método cambie el Estado correctamente en las pruebas unitarias.

Clases y métodos relacionados: Torneo.iniciar(), que valida y actualiza el Estado de la entidad.

HU-T-ADM-03: Finalizar un torneo y entregar premios
Como administrador, quiero finalizar un torneo y registrar al socio ganador, para entregarle el bono de descuento (si es recreativo) o transferirle el pozo acumulado (si es competitivo).

Criterios de aceptación:

- CA-1: Dado un torneo EN_CURSO, cuando el administrador lo finaliza indicando al ganador inscrito, entonces el sistema cambia el estado a FINALIZADO y registra a dicho socio como el ganador oficial.
- CA-2: Dado un torneo RECREATIVO que se finaliza exitosamente, cuando se registra al ganador, entonces el sistema crea y le otorga a este socio un Bono de Descuento del 20%.
- CA-3: Dado un torneo COMPETITIVO que se finaliza exitosamente, cuando se registra al ganador, entonces el sistema transfiere el recaudo total del torneo al saldo del socio.
- CA-4: Dado un torneo en curso, cuando el administrador intenta finalizarlo ingresando el ID de un ganador que no está en la lista de inscritos, entonces el sistema rechaza la operación, indicando que el jugador no pertenece al torneo, y el estado se mantiene EN_CURSO.

Entradas: Identificador del torneo e identificador (login) del socio ganador.
Salidas: Mensaje de confirmación detallando el premio entregado (bono o monto del pozo) o mensaje de error si el usuario no es válido.

Cumplimiento de INVEST: Aporta alto Valor al cerrar el ciclo del evento y gestionar la recompensa (monetaria o de fidelización). Es Estimable y Pequeña ya que unifica el cambio de estado con la delegación del premio. Es Comprobable verificando el saldo o la lista de bonos del socio tras la operación.

Clases y métodos relacionados: La operación centralizada ocurre en Club.finalizarTorneoYEntregarPremio(). Internamente delega la validación de estado e inscritos a Torneo.finalizar(), el cálculo de caja a Torneo.getRecaudoTotal(), y la adjudicación de premios a Socio.recibirBono() y Socio.agregarSaldo().

HU-T-ADM-04: Consultar el historial de los torneos
Como administrador, quiero visualizar un listado con la información básica de todos los torneos del club, para monitorear el estado general de las competencias ofrecidas.

Criterios de aceptación:

- CA-1: Dado que existen torneos creados en el club, cuando el administrador solicita ver el historial, entonces el sistema imprime por consola el ID, el nombre y el estado actual de cada torneo.

Entradas: Ninguna (opción seleccionada en el menú).
Salidas: Listado impreso en consola con los torneos registrados.

Clases y métodos relacionados: Acción exclusivamente de presentación, manejada desde ConsolaAdministrador.listarTorneos(), que accede a la información con Club.getTorneos()

Historia de Usuario Torneos Socio:

- Las historias de usuario describen lo que el socio hace, o puede hacer, con los torneos internos. Cada una sigue el formato rol-acción-beneficio, además los criterios de aceptación son dado-cuando-entonces y cada historia muestra sus entradas y salidas. Adicionalmente cumplen con INVEST, todas son independientes y aportan valor por sí solas.

- **HU-T- SOC-01 – Inscribirse en un torneo:**
    - Como socio, quiero inscribirme en un torneo interno, para participar en las competencias del club.

- **Criterios de Aceptación:**
    - CA-1: Dado un torneo con inscripciones abiertas y cupos libres, cuando el socio se inscribe, entonces queda registrado como participante y los cupos disponibles bajan.
    - CA-2: Dado que el socio tiene un buen ranking en la categoría y quedan cupos preferenciales, cuando el socio se inscribe, entonces ocupa un cupo preferencial.
    - CA-3: Dado un torneo sin cupos disponibles, o , un torneo que ya empezó, cuando el socio se intenta registrar entonces el sistema rechaza la inscripción.
    - CA-4: Dado un torneo competitivo, cuando el socio se inscribe entonces su tarifa de inscripción se suma al recaudo del torneo.

    - Entradas : ID del torneo y el compañero en caso de ser un torneo de dobles
    - Salidas : Confirmación de inscripción con el nuevo cupo y la tarifa , o , el rechazo y la razón de rechazo.

- **HU-T-SOC-02 – Retirarse de un torneo:**
    - Como socio, quiero retirarme de un torneo antes de que empiece para asi liberar mi cupo si ya no puedo participar.

- **Criterios de Aceptación:**
    - CA-1: Dado un socio inscrito en un torneo que todavía no ha iniciado, cuando se retira, entonces deja de ser participante y el cupo queda libre.
    - CA-2: Dado un torneo que ya está en curso o que ya finaliza, cuando el socio intenta retirarse, entonces el sistema rechaza el retiro.

    - Entradas: El ID del torneo
    - Salidas: La confirmación del retiro o el motivo del rechazo

- **HU-T-SOC-03 – Consultar el estado y el historial de un torneo:**
    - Como socio , quiero consultar el estado y el historial de un torneo para seguir como va ver los resultados

- **Criterios de Aceptación:**
    - CA-1: Dado un torneo en cualquier estado, cuando el socio lo consulta, entonces el sistema muestra su estado actual.
    - CA-2: Dado un torneo que ya tiene encuentros jugados, cuando el socio busca su historial, entonces el sistema muestra los participantes y los encuentros con sus resultados y posiciones.

    - Entradas: ID torneo
    - Salidas: El estado del torneo y su historial

- **HU-T-SOC-04 – Recibir y usar el bono de descuento**
    - Como socio, quiero recibir un bono de descuento al ganar un torneo recreativo y usarlo en una compra futura, para ahorrar en la tienda o la cafetería del club.

- **Criterios de Aceptación:**
    - CA-1: Dado un socio que gana un torneo recreativo, cuando el torneo se acaba, entonces el socio recibe un bono de descuento el cual queda guardado.
    - CA-2: Dado un socio con un bono disponible, cuando el socio hace una compra y decide usar el bono, entonces el total de la compra baja según el porcentaje del bono y el bono queda marcado como usado.
    - CA-3: Dado un bono que ya fue usado, cuando el socio intenta usarlo otra vez, entonces el sistema lo rechaza.
    - CA-4: Dada una compra en la que ya se aplicaron o puntos o un código de descuento, cuando el socio intenta usar el bono, entonces el sistema lo rechaza, ya que no son acumulables.

    - Entradas: Producto, la cantidad y si se quiere utilizar el bono
    - Salidas: El total de la compra con el respectivo descuento aplicado o el motivo del rechazo.

Historias de Usuario: Torneos del Empleado

Las historias de usuario de los empleados describen las condiciones bajo las cuales pueden participar en los torneos individuales del club. Se establece que su participación no puede interferir con los turnos laborales, no deben pagar inscripción ni recibir premios.

HU-T-EMP-01: Inscribirse en un torneo individual

Como empleado, quiero inscribirme gratuitamente en un torneo individual, cuando no esté de turno, para participar en las actividades deportivas del club sin incumplir mis responsabilidades laborales.

Criterios de aceptación:

- CA-1: Dado un empleado que no tiene un turno laboral superpuesto con el horario de un torneo individual y cumple las condiciones de inscripción, cuando solicita su inscripción, entonces el sistema lo registra como participante sin cobrarle una tarifa.
- CA-2: Dado un empleado cuyo turno laboral coincide con el horario del torneo, cuando intenta inscribirse, entonces el sistema rechaza la inscripción e informa que existe un conflicto de horarios.
- CA-3: Dado un empleado que ya está inscrito en un torneo, cuando intenta inscribirse nuevamente, entonces el sistema rechaza la operación para evitar una inscripción duplicada.
- CA-4: Dado un empleado inscrito en un torneo, cuando se consulta su inscripción, entonces esta conserva la condición de participación gratuita y sin derecho a premio.

Entradas:

- Identificación del empleado autenticado.
- Identificador del torneo individual.
- Horario del torneo y turnos laborales asignados, consultados por el sistema.

Salidas:

- Confirmación de la inscripción, indicando que es gratuita y no otorga derecho a premio.
- Mensaje de rechazo cuando exista un cruce de horario, una inscripción duplicada o se incumplan las condiciones de participación.

Cumplimiento de INVEST:

- La historia es independiente porque se enfoca en la inscripción del empleado. Es negociable porque permite ajustar la forma de presentar mensajes y opciones. Además, es valiosa porque permite participar sin afectar las obligaciones laborales. También es estimable y pequeña porque está delimitada a una operación concreta. Por último, es comprobable porque cada criterio de aceptación puede validarse mediante pruebas automatizadas.

Clases y métodos relacionados:

- La operación se realiza en Club.inscribirEmpleadoEnTorneo(), utilizando la información del empleado y sus turnos laborales. La clase InscripcionTorneoEmpleado representa la participación. La consola ConsolaEmpleado permite solicitar la inscripción y presenta los mensajes de confirmación o rechazo.

Pruebas:

- La clase PersistenciaTest, ubicada en el paquete de pruebas, incluye verificaciones relacionadas con inscripción gratuita, turnos superpuestos, duplicados y recuperación de información.

Persistencia de datos

El Club Deportivo Séneca implementa persistencia con archivos de texto plano almacenados en la carpeta data, en la raíz del proyecto.

Los archivos utilizan UTF-8 y separan sus campos con punto y coma (;). Cada línea representa un registro y las relaciones entre las entidades se conservan con identificadores, como el login del usuario, el identificador del torneo o el código del producto.

El sistema utiliza dos clases principales:

- PersistenciaClub: coordina el guardado y la carga del club, incluyendo usuarios, turnos, torneos, inscripciones y bonos.
- PersistenciaDatosClub: complementa el almacenamiento de ubicaciones, productos, inventario, sugerencias, disciplinas individuales, inscripciones deportivas y reservas.

Archivos Usados:

Archivo	Columnas, en orden
usuarios.txt	tipoUsuario;login;password;nombre;fechaNacimiento;puntos
turnos.txt	loginEmpleado;diaSemana;horaInicio;horaFin
torneos.txt	idTorneo;nombre;tipo;modalidad;cupoMaximo;tarifa;fecha;horaInicio;horaFin;estado;cuposPreferencialesDisponibles;cuposEstandarDisponibles;recaudoTotal
inscripciones_empleados.txt	idTorneo;loginEmpleado
inscripciones_socios.txt	idTorneo;loginSocio
bonos.txt	loginSocio;porcentaje;usado
ubicaciones.txt	nombreUbicacion
productos.txt	tipoProducto;codigo;nombre;precio;categoria

	OTemperatura;alergenos
existencias.txt	codigoArticulo;nombreUbicacion;cantidad
sugerencias.txt	fecha;descripcion;loginEmpleado;codigo;nombre;precio;esSnack;estado;alergenos
disciplinas.txt	tipoDisciplina;maxInstalacionesSimultaneas
inscripciones_individuales.txt	tipoDisciplina;loginSocio;nivel;puntajeRanking

Tratamiento de alérgenos

Para los productos y sugerencias que contienen alérgenos, los valores se almacenan dentro de una misma columna, separados mediante comas. En las sugerencias, el sistema distingue entre una lista de alérgenos declarada y la ausencia de una declaración. Cuando el empleado no ha declarado los alérgenos, se almacena el valor SIN_DECLARAR. Esto permite diferenciar una sugerencia cuyo contenido fue revisado y declarado sin alérgenos de otra en la cual todavía no existe esa declaración. La distinción se conserva al recuperar el estado de la aplicación.

Orden de carga

La carga se realiza en un orden que permite reconstruir correctamente las relaciones entre objetos:

1. Usuarios.
2. Turnos laborales, asociándolos con empleados existentes.
3. Torneos.
4. Inscripciones de empleados.
5. Inscripciones de socios.
6. Estados y cupos de torneos.
7. Bonos de descuento asociados a socios.
8. Ubicaciones.
9. Productos.
10. Existencias, asociando artículos y ubicaciones.
11. Sugerencias, asociándolas con sus empleados.
12. Disciplinas individuales.
13. Inscripciones individuales y rankings.
14. Reservas deportivas.

El orden evita crear referencias a entidades que aún no han sido reconstruidas.

Carga y guardado desde las consolas:

Las tres consolas comparten la clase ConsolaBase, encargada de coordinar la autenticación, la lectura validada de datos y el ciclo de persistencia. Al iniciar una consola se ejecuta el método cargar() de PersistenciaClub, que reconstruye un nuevo objeto Club con los datos almacenados en data. Después, el usuario debe ingresar sus credenciales ya que el sistema utiliza Club.autenticar() y comprueba que el usuario corresponda al rol de la consola seleccionada. Al finalizar normalmente el menú, ConsolaBase ejecuta guardar(club), y esta operación actualiza los archivos y conserva los cambios de los módulos cubiertos por la persistencia.

Validaciones y manejo de errores

La implementación verifica la cantidad de columnas de los registros, rechaza ids duplicados durante la carga y comprueba que las referencias corresponden a objetos existentes. También rechaza campos que contengan punto y coma o saltos de línea, evitando que estos alteren la estructura del archivo.

Cuando se encuentra información inválida durante la recuperación, las clases de persistencia generan una excepción de entrada o salida, que la consola comunica mediante un mensaje de error.

Alcance de la persistencia

La implementación conserva los datos de los trece archivos descritos arriba, incluyendo los puntos de los usuarios, los estados de las sugerencias, las inscripciones a torneos, los bonos usados y disponibles, los rankings individuales y las reservas. El almacenamiento documentado no incluye historiales completos de ventas, resultados detallados de partidos ni registros de asistencia a entrenamientos. Por lo tanto, la conservación del estado se refiere a las entidades implementadas en los trece archivos anteriores.

Uso de Inteligencia Artificial:

- **Simon Hani :**
    - Utilice Claude para primero que todo entender el proyecto, entender que pide el proyecto y la rúbrica, revisar todas las historias de usuario de banco además de analizar los comentarios que nos dejaron en el proyecto 1.
    - Despues lo use para antes de empezar todo el proyecto para separar el proyecto en 3 partes las cuales incluyen todo lo nuevo que teníamos que hacer y todo lo que nos faltó de la entrega del proyecto 1
    - Luego le pedí que me ayudara a como empezar, donde me dijo que creará la clase “Club”.
    - Después le pedí que me enseñara la estructura de los tests y que me guiara para ir creándolos.

- **Gabriella Siman**

    - Para el desarrollo del proyecto, emplee la inteligencia artificial como herramienta facilitadora más que generadora. Primero, la utilice para comprender cómo dividir adecuadamente las tareas del proyecto y que debía hacer en mi parte individual. Le pedí que me diera puntos de mejora específicos basados en los comentarios de la última entrega. Con esto mejoré aquellas partes faltantes de la primera entrega. En el resto del proyecto le pedí que corrigiera mis tests y le pedí guías de construcción para elementos como los Junits y las consolas, las cuales no sabía programar por mi cuenta para que funcionaran adecuadamente.
- Tomas Vargas

    Empleé la IA para diseñar la base de la interfaz ConsolaAdministrador, garantizando la separación estricta entre la lógica de negocio y la presentación. Gemini me ayudo conen el manejo de excepciones (try-catch) para capturar errores sin exponer la lógica interna. También para finalizar este doc y para seguir con las incorporaciones de codigo y analisis, que cuando se hicieran merges todo siguiera sirviendo